

The background image is a dark, atmospheric server room. Rows of server racks are illuminated with a vibrant green light. In the foreground, a control console with multiple screens and buttons is visible, also glowing with green light. One screen displays a world map. The overall aesthetic is high-tech and digital.

Sesión 4: PostgreSQL y Persistencia de Datos

De listas en memoria a una base de datos relacional profesional. Hoy conectamos FastAPI con PostgreSQL a través de SQLAlchemy, diseñamos nuestro modelo de datos, implementamos un CRUD completo y aplicamos el Repository Pattern con migraciones gestionadas por Alembic.

De memoria temporal a persistencia real

El problema de la sesión anterior

Con `tickets = []`, los datos desaparecen al reiniciar FastAPI. No hay persistencia, ni integridad, ni soporte para múltiples instancias.

La solución: PostgreSQL

PostgreSQL garantiza que un ticket creado el lunes siga existiendo tras reinicios, apagones y consultas concurrentes.

- Persistencia duradera y fiable
- Restricciones e integridad referencial
- Transacciones ACID
- Índices y consultas complejas

MODELADO DE DATOS

Antes del código: diseñemos los datos

USER

id · name · email

CATEGORY

id · name

TICKET

id · title · priority · status · requester_id · category_id · created_at

Relaciones 1:N: un usuario crea varios tickets y una categoría agrupa varios tickets. Las **primary keys** identifican registros; las **foreign keys** conectan entidades.

SOFTWARE DESIGN PLATFORM ERD v2.1



NOTES

- PRIMARY KEYS: id (UUID)
- ALL DATES IN UTC
- SOFT DELETE: is_deleted
- TENANT ISOLATION
- AUDITABLE ENTITIES

KEY MODULES

- AUTH & USER MANAGEMENT
- BILLING & PAYMENTS
- PROJECT MANAGEMENT
- NOTIFICATIONS
- REPORTING & ANALYTICS

RELATIONSHIP GUIDE

1 — N
(one to many)

1 — 1
(one to one)

N — N
(many to many)

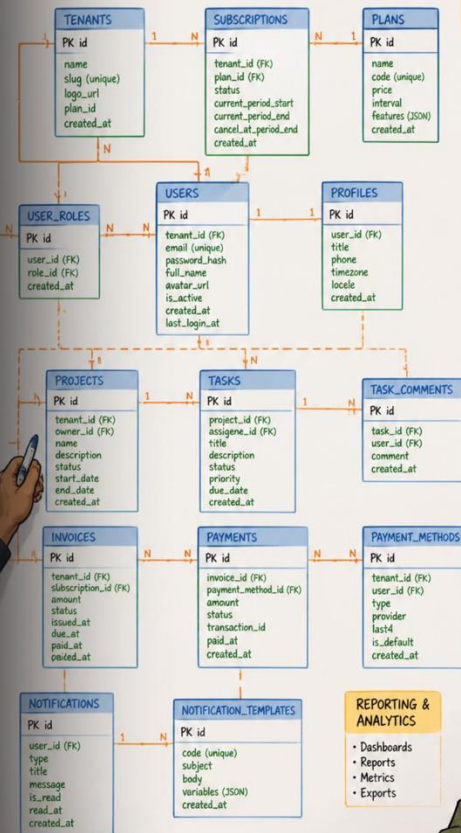
— (optional)

TODO

- ☐ Review indexing strategy
- ☐ Add audit log entity
- ☐ Define cascade rules
- ☐ Data retention policy
- ☐ API contract review

ASSUMPTIONS

- Multi-tenant SaaS
- Tenant scoped data
- Users belong to one tenant
- Roles are tenant scoped



DESIGN
FOR SCALE

Lo que SQLAlchemy hará por nosotros

Aunque usemos ORM, comprender SQL es imprescindible para depurar, optimizar y tomar decisiones informadas.

INSERT

```
INSERT INTO tickets (title, priority) VALUES ('Error login', 'Alta');
```

SELECT + WHERE

```
SELECT * FROM tickets WHERE priority = 'Alta';
```

UPDATE / DELETE

```
UPDATE tickets SET status = 'Cerrado' WHERE id = 1;
```

JOIN

```
SELECT t.title, c.name FROM tickets t JOIN categories c ON t.category_id = c.id;
```

🔑 **Clave:** ORM no significa «no necesito saber SQL».



Conectando FastAPI con PostgreSQL

Dependencias y conexión

```
python -m pip install sqlalchemy
python -m pip install "psycopg[binary]"
```

```
DATABASE_URL = (
    "postgresql+psycopg://postgres:postgres"
    "@localhost:5432/helpdesk_db"
)
```

```
engine = create_engine(
    DATABASE_URL, pool_pre_ping=True
)
```

Session y dependencia FastAPI

```
SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False
)
```

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

❏ Inyecta `Depends(get_db)` en tus rutas para obtener una sesión por petición.

ORM

De una tabla PostgreSQL a una clase Python

```
class Ticket(Base):  
    __tablename__ = "tickets"  
  
    id: Mapped[int] = mapped_column(primary_key=True)  
    title: Mapped[str] = mapped_column(String(120))  
    description: Mapped[str]  
    priority: Mapped[str] = mapped_column(String(20))  
    status: Mapped[str] = mapped_column(  
        String(30), default="Pendiente"  
    )  
    requester_id: Mapped[int] = mapped_column(  
        ForeignKey("users.id")  
    )  
    category_id: Mapped[int] = mapped_column(  
        ForeignKey("categories.id")  
    )  
    created_at: Mapped[datetime] = mapped_column(  
        DateTime(timezone=True), server_default=func.now()  
    )
```

Clase Python

• Tabla

Objeto

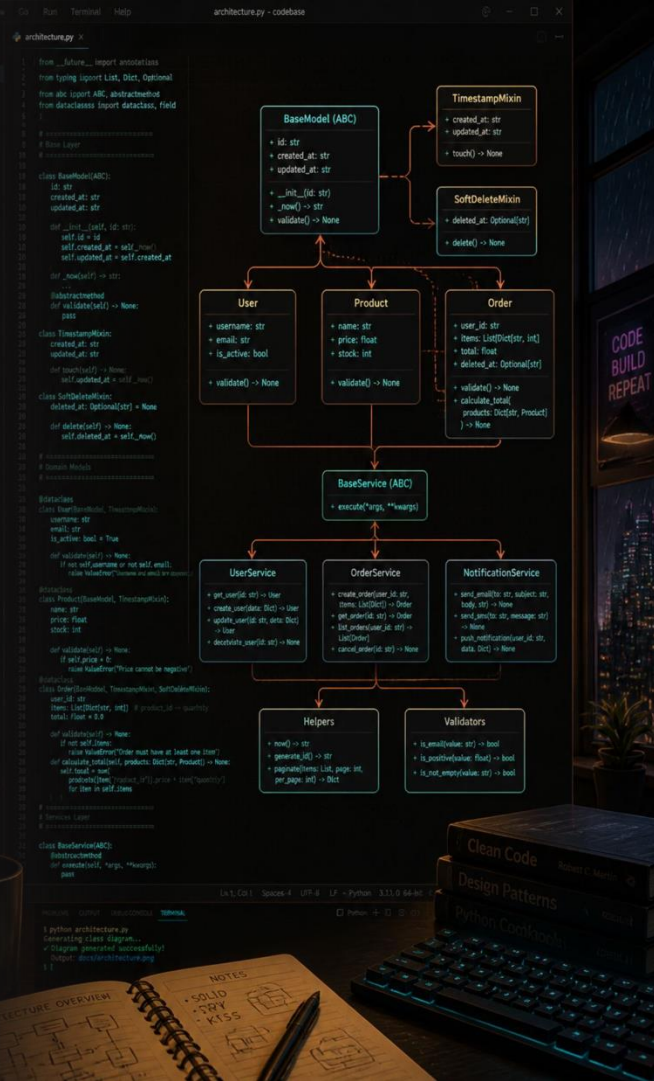
• Registro

Atributo

• Columna

Mapped / ForeignKey

• Tipo y relación



Persistiendo las operaciones de nuestra API

1

CREATE

```
ticket = Ticket(title=data.title, priority=data.priority)
db.add(ticket)
db.commit()
db.refresh(ticket)
```

2

READ

```
from sqlalchemy import select
stmt = select(Ticket)
tickets = db.scalars(stmt).all()
ticket = db.get(Ticket, ticket_id)
```

3

UPDATE

```
ticket.status = "Cerrado"
db.commit()
db.refresh(ticket)
```

4

DELETE

```
db.delete(ticket)
db.commit()
```

No queremos trabajar solamente con IDs

Modelo `User`

```
class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(
        primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    tickets: Mapped[list["Ticket"]] = relationship(
        back_populates="requester")
```

Modelo `Ticket`

```
class Ticket(Base):
    __tablename__ = "tickets"
    requester_id: Mapped[int] = mapped_column(
        ForeignKey("users.id"))
    requester: Mapped["User"] = relationship(
        back_populates="tickets")
```

- ❑ Ahora podemos usar `ticket.requester.name` o `user.tickets` directamente en Python.

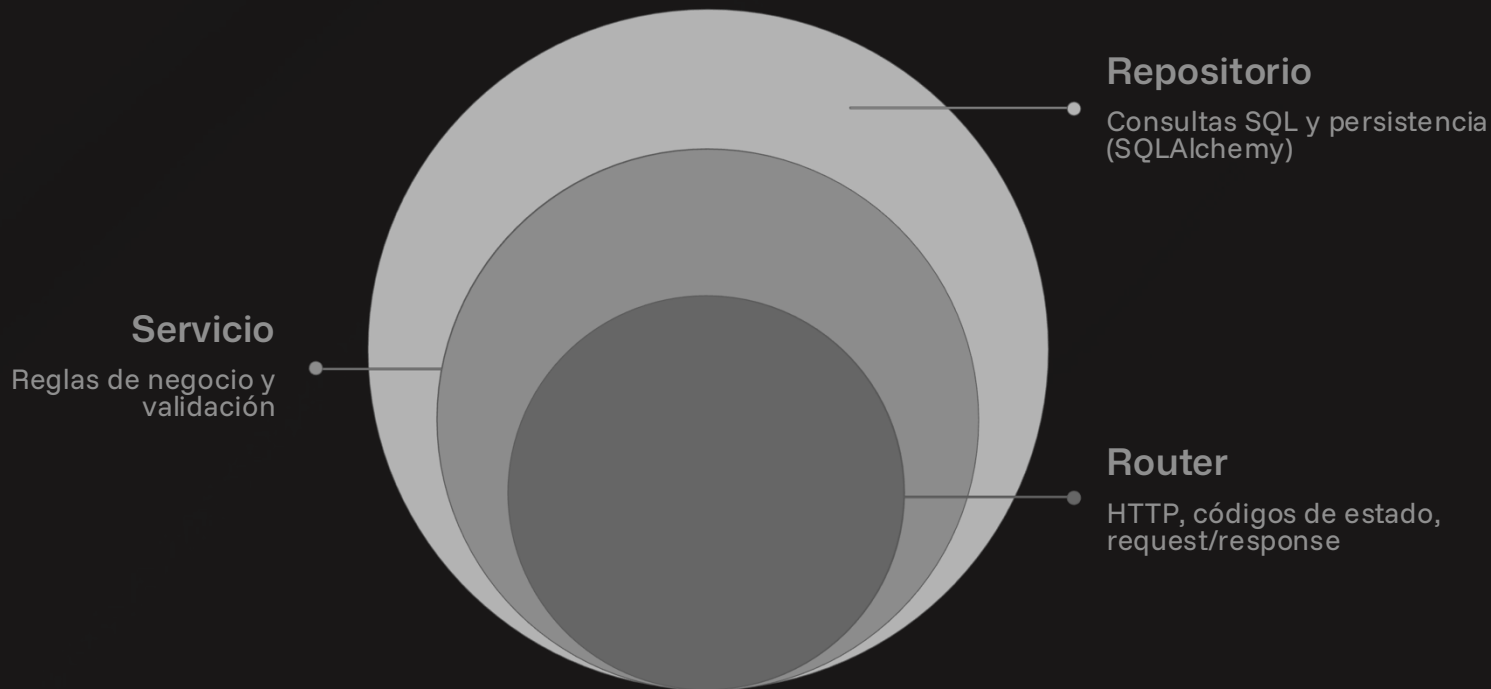
ForeignKey

Define la relación en PostgreSQL

relationship()

Habilita la navegación entre objetos Python

Repository Pattern: separar HTTP del acceso a datos



El Repository Pattern introduce una capa intermedia que aísla la lógica de acceso a datos del resto de la aplicación, facilitando el testing y el mantenimiento.

✗ Mala práctica

Mezclar consultas SQL directamente en el router dificulta el testing y viola el principio de responsabilidad única.

Alembic y buenas prácticas de persistencia

Migraciones con Alembic

```
alembic init migrations
alembic revision --autogenerate \
    -m "create helpdesk tables"
alembic upgrade head
```

Buenas prácticas

- Credenciales en variables de entorno
- Transacciones y restricciones en BD
- Índices según consultas reales
- Separar persistencia de lógica de negocio
- Cerrar sesiones correctamente
- Manejar errores de base de datos

Las migraciones permiten evolucionar el esquema **sin perder datos**.

1

Sesión 3

FastAPI + lista en memoria

2

Sesión 4

FastAPI + SQLAlchemy + PostgreSQL

3

Sesión 5

¿Una aplicación o varios servicios?

❓ Si mañana añadimos `assigned_at`, ¿debemos borrar y recrear la base de datos?